Algorithms Portfolio

Curated algorithmic solutions with correctness and complexity analysis

---

Ryan McMillan

MCompSci (cand.) – University of New England

C++ primary • Python secondary

February 15, 2026

License

Glossary

Amortized analysis
An analysis technique that considers the average cost of operations over a sequence of operations, even if individual operations may be expensive.

Invariant A property that remains true before and after each iteration of an algorithm or loop.

In-place algorithm
An algorithm that transforms its input using only a constant amount of additional memory.

Loop invariant
An invariant associated with a loop, used to prove correctness via initialization, maintenance, and termination.

Time complexity
An asymptotic measure of how an algorithm's running time grows as a function of input size.

Contents

Glossary **2**

1 Introduction **4**
- 1.1 How to Use This Book . . . 4
- 1.2 Motivation . . . 4
- 1.3 Acknowledgements . . . 4

2 Arrays & Sequences **6**
- 2.1 Core Ideas . . . 6
- 2.2 Common Pitfalls . . . 6
- 2.3 Primer - Arrays & Sequences . . . 6
- 2.4 Easy . . . 8
  - 2.4.1 Plus One . . . 8

3 Strings & Parsing **10**
- 3.1 Core Ideas . . . 10
- 3.2 Common Pitfalls . . . 10
- 3.3 Easy . . . 10
  - 3.3.1 Valid Palindrome . . . 10

4 Linked Structures **13**
- 4.1 Core Ideas . . . 13
- 4.2 Common Linked Structure Variants . . . 13
- 4.3 Common Pitfalls . . . 13
- 4.4 Medium . . . 13
  - 4.4.1 Add Two Numbers . . . 13

1 Introduction

This document is a curated portfolio of algorithmic solutions. Each selected problem includes:

- clear problem restatements,
- key invariants / observations,
- pseudocode,
- correctness sketches,
- complexity analysis,
- C++ (primary) and Python (secondary) implementations.

The focus is on clarity, correctness, and reasoning.

1.1 How to Use This Book

This book is not a tutorial on data structures and algorithms, nor does it attempt to replace formal coursework or foundational texts. Instead, it serves as a refresher and practical reference, organised around LeetCode-style problems that commonly appear in technical interviews and applied programming contexts.

The emphasis is on recognising patterns, recalling standard techniques, and reviewing concise, correct implementations. Explanations focus on the idea being exercised and the invariants maintained, rather than on full theoretical development.

Readers are assumed to already be familiar with fundamental data structures and algorithmic concepts. The book is intended to be used for revision, reinforcement, and interview preparation, particularly when revisiting topics after time away from formal study.

Problems include an optional **Author's Thinking** block, the authors plain language thinking and problem framing, which is excluded from upstream exemplar builds. Use this book as an example, to understand how to formally reason with, and structure algorithm and data structures problems.

1.2 Motivation

This portfolio was created as a structured, self-directed study of fundamental algorithms and data structures. While my formal coursework emphasizes applied computer science and systems-level engineering, it does not include a dedicated course focused on algorithmic problem-solving in the style commonly expected in technical interviews and quantitative software roles.

To address this, I undertook a systematic review of core algorithmic techniques, organizing problems by data structure and algorithmic paradigm, and documenting each solution with an explicit algorithm description, correctness argument, and complexity analysis. The goal is not only to arrive at correct implementations, but to practice communicating algorithmic reasoning clearly and rigorously.

This document serves both as a personal reference and as evidence of deliberate practice in algorithmic reasoning beyond formal coursework.

Many of the problems included here are sourced from commonly used interview problem sets, and were selected to emphasize invariant-based reasoning, edge case analysis, and asymptotic performance trade-offs.

Informal reasoning notes are included selectively to document thought process and learning progression; these sections are omitted in derived exemplar or presentation-focused versions of this material.

1.3 Acknowledgements

This work was informed by a number of open educational resources that emphasize rigorous algorithmic reasoning and clear exposition. In particular, the structure and approach to cor-

rectness arguments and asymptotic analysis were influenced by *MIT OpenCourseWare 6.006: Introduction to Algorithms*.

Additional reference was made to publicly available problem discussions, editorials, and textbooks where appropriate. All implementations, explanations, and written analyses in this document are my own, and any external influences are acknowledged at a conceptual level rather than reproduced directly.

2 Arrays & Sequences

Arrays are the foundational data structure in most algorithmic problems. They provide constant-time indexed access and strong cache locality, but require careful reasoning about indices, bounds, and in-place mutation.

This chapter focuses on recognizing array-based problem patterns and maintaining clear invariants over contiguous memory.

2.1 Core Ideas

- Index-as-state reasoning
- In-place mutation vs auxiliary storage
- Prefix and suffix accumulation
- Simulation of elementary operations (e.g. addition, carry propagation)

2.2 Common Pitfalls

- Off-by-one errors at boundaries
- Unintended data overwrites during in-place updates
- Assuming sorting or uniqueness without verifying constraints
- Over-allocating space when a single pass suffices

.

2.3 Primer - Arrays & Sequences

What It Is

sed.). An array is a *contiguous* block of memory storing elements of the same type, supporting $O(1)$ access by index. A dynamic array (e.g. C++ `std::vector`, Python `list`) grows by allocating a larger block and copying elements when capacity is exceeded, providing *amortized* $O(1)$ append. A *sequence* generalises the idea to any indexable or iterable ordered collection; here we emphasise contiguous arrays.

Recognition Patterns

.

- Index math across contiguous elements (prefix/suffix, window, ranges).
- Constraints emphasising in-place mutation, $O(1)$ extra space, single pass
- Repeated accumulation or difference of segments (prefix sums/scans).
- Local adjacency decisions: next/previous comparison, runs, peaks/valleys

Core Operations

```
access(A, i):
    return A[i] // O(1)

traverse(A):
  for i from 0 to n-1:
    visit(A[i])

insert_at(A, i, x):
  grow A by 1
  for j from n-1 downto i:
    A[j+1] = A[j]
  A[i] = x

erase_at(A, i):
  for j from i to n-2:
    A[j] = A[j+1]
  shrink A by 1

prefix_sums(A):
  P[0] = 0
  for i from 0 to n-1:
    P[i+1] = P[i] + A[i]
  return P

two_pointers_sorted(A, target):
  i = 0; j = n - 1
  while i < j:
    s = A[i] + A[j]
    if s == target: return (i, j)
    if s < target: i = i + 1 else: j = j - 1
  return not found

sliding_window(A):
  l = 0
  for r from 0 to n-1:
    add(A[r])
    while window_invalid():
      remove(A[l])
      l = l + 1
    update_answer()
```

Listing 1: Arrays: Core Operations (pseudocode)

Invariants & Correctness

Array scans typically maintain index-based invariants: (i) after i steps, positions $[0..i)$ have been processed, (ii) two-pointer: indices satisfy $0 \leq i \leq j < n$ and a monotone move (left increases or right decreases) prevents rework, (iii) sliding window: the maintained sub-array $[l, r)$ satisfies a property (e.g. distinct elements, sum $\leq K$); expansion and contraction preserve the property.

Complexity

- Access: time $O(1)$; Traversal: $O(n)$
- Insert/delete at end (dynamic array): amortised $O(1)$, at index $O(n)$
- Prefix sums / scan: $O(n)$ time, $O(n)$ extra space (or $O(1)$ in place if possible)
- Two-pointers / sliding window: typically $O(n)$ if each pointer advances monotonically.

Common Pitfalls

- Off-by-one at boundaries; confusing inclusive/exclusive ranges.
- Forgetting to update all window state on both expand and shrink
- Overflow in sums/producs; use wider types for accumulation.
- Accidental $O(n^2)$ from repeated slicing/copying in inner loops.
- Assuming sortedness/uniqueness without verifying constraints.

)

2.4 Easy

The following problems introduce fundamental array manipulation techniques. The emphasis is on establishing and maintaining simple invariants while iterating through the array.

2.4.1 Plus One

Difficulty: Easy **Tags:** Arrays, Simulation **ID:** LC-066

Problem. Given a non-empty array of digits representing a non-negative integer, increment the integer by one and return the resulting array of digits.

The most significant digit is stored at the beginning of the array, and each digit is in the range $[0, 9]$.

Key idea. Incrementing the number requires simulating elementary addition from right to left. Carry propagation only affects trailing digits equal to nine; once a digit strictly less than nine is incremented, the process terminates.

Author's Thinking.

> My initial instinct was to convert the digit array into an integer, add one, and convert it back. This approach fails due to potential overflow and violates the problem constraints.
>
> What simplified the reasoning was treating the operation as grade-school addition and maintaining the invariant that all digits to the right of the current index are already correct after carry handling.

Algorithm.

```
for i from n - 1 downto 0:
    if digits[i] < 9:
        digits[i] = digits[i] + 1
        return digits
    digits[i] = 0

prepend 1 to digits
return digits
```

Listing 2: Pseudocode

Correctness sketch. We iterate from the least significant digit toward the most significant digit. At each iteration, the invariant maintained is that all digits to the right of the current index represent the correct result after increment and carry propagation.

If a digit less than nine is encountered, incrementing it produces the correct result and no further carry is required. If all digits are nine, each is set to zero and a leading one is prepended, yielding the correct incremented number. Thus, the algorithm always returns the correct result.

Complexity. The algorithm runs in $O(n)$ time, where n is the number of digits. It uses $O(1)$ additional space beyond the output array.

C++ Implementation

```cpp
/*
* LC-066 - Plus One
*
*/

#include <iostream>

class Solution {
public:
    vector<int> plusOne(vector<int>& digits) {
        for(int i = digits.size(); i-- > 0 ; ) {
            if (digits[i] != 9) {
                digits[i]++;
                return digits;
            }
            else {
                digits[i] = 0;
            }
        }

        vector<int> newDigits(digits.size() + 1);
        newDigits[0] = 1;
        for (int j = 1; j < newDigits.size(); j++) {
            newDigits[j] = digits[j-1];
        }

        return newDigits;
    }
};
```

Listing 3: C++ Implementation

Python Implementation

```python
# LC-066 Plus One

from typing import List

def plusOne(digits: List[int]) -> List[int]:
    for i in range(len(digits) - 1, -1, -1):
        if digits[i] < 9:
            digits[i] += 1
            return digits
        digits[i] = 0

    return [1] + digits
```

Listing 4: Python Implementation

3 Strings & Parsing

Strings are a fundamental data structure used to represent text and sequences of characters. They often require specialized algorithms for efficient manipulation, searching, and pattern matching. This chapter focuses on recognizing string-based problem patterns and maintaining clear invariants over character sequences.

3.1 Core Ideas

- Two-pointer techniques for substring problems
- Sliding window approaches for dynamic substring analysis
- Character classification and normalisation
- Maintaining loop invariants over indicies
- Incremental Parsing
- In-place vs Auxiliary Storage

3.2 Common Pitfalls

- Off-by-one errors at boundaries
- Incorrect handling of empty strings or single-character strings
- Failing to advance pointers correctly when skipping characters
- Unnecessary string copying leading to increased time/space complexity
- Assumptions about character sets (e.g., ASCII vs Unicode)

3.3 Easy

The following problems introduce fundamental string parsing and manipulation techniques. The emphasis is on establishing and maintaining simple invariants while iterating through the array.

3.3.1 Valid Palindrome

Difficulty: Easy **Tags:** Strings, Two Pointers **ID:** LC-125

Problem. Given a string s, determine whether it is a palindrome when considering only alphanumeric characters and ignoring case. Non-alphanumeric characters (punctuation/whitespace) are ignored. After filtering, the empty string is considered a palindrome.
Key idea. We need to compare the string from both ends; a two-pointer approach lets us scan inward while skipping non-alphanumeric characters, achieving $O(n)$ time and $O(1)$ extra space.

Author's Thinking.

> <Informal reasoning, failed approaches, intuition-building, or mental models. This section is intentionally exploratory and may be omitted in promoted or exemplar versions of the book.>

Algorithm.

```
initialise left = 0, right = length(s) - 1
while left < right:
    if s[left] is not alphanumeric:
        left = left + 1
        continue
    if s[right] is not alphanumeric:
        right = right - 1
        continue
    if lowercase(s[left]) != lowercase(s[right]):
        return false
    left = left + 1
    right = right - 1
return true
```

Listing 5: Pseudocode

Correctness sketch. At the start of each iteration, all alphanumeric characters strictly outside the interval $[left, right]$ have already been matched (case-insensitively) with their corresponding partner on the other side. Skipping a non-alphanumeric character is safe because such characters are ignored by the problem definition. If $\text{lower}(s[left]) \neq \text{lower}(s[right])$, then an unmatched pair exists, so the string cannot be a valid palindrome and we return `False`. When the loop terminates ($left \geq right$), every relevant pair has been verified; a middle character (odd length after filtering) does not affect palindromicity.

Complexity. Each pointer moves inward at most n times, so time complexity is $O(n)$. The algorithm uses $O(1)$ extra space (in-place scanning; no filtered string constructed).

Python Implementation

```python
# LC 125. Valid Palindrome

class Solution:
    def isPalindrome(self, s: str) -> bool:
        """
        Determines if a given string is a valid palindrome, considering only
        alphanumeric characters and ignoring case sensitivity.

        Args:
            s (str): The input string.

        Returns:
            bool: True if the string is a palindrome, False otherwise.
        """
        # Initialise two pointers
        left = 0
        right = len(s)-1

        # Iterate until the left and right pointers meet
        while (left < right):
            if not s[left].isalnum():
                left += 1
                continue
            if not s[right].isalnum():
                right -= 1
                continue
            if s[left].lower() != s[right].lower():
                return False

            left += 1
```

```
            right -= 1

        return True
```

Listing 6: Python Solution: LC-125 Valid Palindrome

4 Linked Structures

Linked structures are **pointer-based** data structures where logical order is defined by explicit **references** rather than **contiguous memory** layout. They sacrifice constant-time indexed access in exchange for flexible insertion, deletion, and natural modeling of sequential processes.

This chapter focuses on recognizing **pointer-driven** problem patterns and maintaining correct **traversal invariants** over **node-based** structures.

4.1 Core Ideas

- **Pointer-as-state** reasoning
- **Sequential traversal** instead of indexed access
- Explicit handling of **null termination**
- Incremental construction using moving **cursors** (e.g. **dummy head** nodes)

4.2 Common Linked Structure Variants

Linked structures appear in several common forms, each with distinct properties and use cases:

- **Singly Linked List** — each node references the next node only
- **Doubly Linked List** — each node references both next and previous nodes
- **Circular Linked List** — the terminal node references an earlier node
- **Sentinel-Based List** — uses a non-data **sentinel node** to simplify edge cases

4.3 Common Pitfalls

- Losing access to the **head node** during traversal or mutation
- Dereferencing **null pointers**
- Incorrect **loop termination conditions**
- Conflating **in-place modification** with **new list construction**

4.4 Medium

The following problems build on basic linked-structure traversal and introduce **stateful pointer algorithms**, where correctness depends on maintaining multiple interacting invariants simultaneously. Rather than simple scans, these problems require coordinating traversal with auxiliary state (such as a **carry**, window offset, or delayed reference) while constructing or modifying linked structures.

The emphasis is on:

- Maintaining multiple **traversal invariants**
- Coordinating pointer movement with additional state
- Incremental construction using **dummy heads** and **cursors**
- Correct handling of **null termination** across uneven structures

4.4.1 Add Two Numbers

Difficulty: Medium **Tags:** Linked Lists, Math, Carry Propagation **ID:** LC-002

Problem. Given two non-empty linked lists representing two non-negative integers, where each node stores a single digit and the digits are arranged in reverse order (least significant digit first), compute the sum of the two integers. Return the result as a new linked list in the same reverse-order format.

Key idea. Because the digits are stored in reverse order, the lists can be traversed from least significant to most significant digit, making the problem a direct simulation of elementary addition. At each step, the current digits and any incoming **carry** determine the next output digit and updated carry. A **dummy head** node anchors the result list, while a moving **cursor** appends newly computed digits without losing access to the list's beginning. Traversal continues until both input lists reach **null termination** and no carry remains, maintaining the **invariant** that all previously processed digits in the result list are correct.

Author's Thinking.

A first brute-force idea is to traverse each list, reconstruct the two integers, add them, and then convert the sum back into a linked list. However, this approach is awkward in practice: the numbers may exceed native integer ranges, and it performs extra passes and conversions that are unnecessary.

A better approach is to simulate column-wise addition directly on the linked lists. Since the digits are stored least-significant-first, we can traverse both lists once, propagate a carry, and build the result list incrementally.

Algorithm.

```
dummy = new Node(0)
cur = dummy
carry = 0

while l1 != null OR l2 != null OR carry != 0:
    x = (l1 != null) ? l1.val : 0
    y = (l2 != null) ? l2.val : 0

    total = x + y + carry
    digit = total mod 10
    carry = total div 10

    cur.next = new Node(digit)
    cur = cur.next

    if l1 != null: l1 = l1.next
    if l2 != null: l2 = l2.next

return dummy.next
```

Listing 7: Pseudocode

Correctness sketch. At the start of each iteration of the loop, the result list contains the correct sum of all digit positions processed so far, and the **carry** variable holds the correct carry-out into the next higher digit position. During the iteration, the algorithm adds the current digits (or zero if a list has reached **null termination**) together with the carry, computes the correct output digit, and updates the carry accordingly. Because each iteration advances at least one input pointer or consumes a remaining carry, the loop eventually terminates. When the loop ends, all digit positions and any final carry have been processed, so the resulting linked list represents the correct sum.

Complexity. Let n and m be the lengths of the two input linked lists. Each iteration processes at most one node from each list, and the loop runs for at most $\max(n, m) + 1$ iterations, yielding a time complexity of $O(\max(n, m))$.

The algorithm uses $O(1)$ auxiliary space, as it maintains only a constant number of pointers and a carry variable. The output linked list requires $O(\max(n, m))$ space to store the result.

C++ Implementation

```cpp
/**
 * Definition for singly-linked list.
 * struct ListNode {
```

```cpp
 * int val;
 * ListNode *next;
 * ListNode() : val(0), next(nullptr) {}
 * ListNode(int x) : val(x), next(nullptr) {}
 * ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */

class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        // Create two ListNode pointers to track the results beginning and current
            address.
        ListNode* dummy = new ListNode(0);
        ListNode* current = dummy;

        // Initialise our carry variable to 0
        int carry = 0;

        // Iterate while we have a non null pointer in L1 or L2 or a carry digit
        while (l1 || l2 || carry) {
            // If Ls contain value use that, otherewise 0
            int x = l1 ? l1->val : 0;
            int y = l2 ? l2->val : 0;

            // Perform calculation for result digit and carry
            int total = x + y + carry;
            carry = total / 10;
            int digit = total % 10;

            // Create memory for a new result node, and point to that node
            current->next = new ListNode(digit);
            current = current->next;

            // Traverse the linked lists if there is a valid node
            if (l1) l1 = l1->next;
            if (l2) l2 = l2->next;

        }

        return dummy->next;

    }
};
```

Listing 8: C++ Solution: LC-002 Add Two Numbers

Python Implementation

```python
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next

class Solution:
    def addTwoNumbers(self, l1: Optional[ListNode], l2: Optional[ListNode]) ->
        Optional[ListNode]:
```

```python
        # Initialise output LinkedList, and carryover integer
        current = ListNode(0)
        dummy = current
        carry = 0

        # Traverse both lists while digits or carry remain
        while(l1 or l2 or carry):
            x = l1.val if l1 else 0
            y = l2.val if l2 else 0

            total = x + y + carry
            digit = total % 10
            carry = total // 10

            # Append current result digit and advance
            current.next = ListNode(digit)
            current = current.next

            l1 = l1.next if l1 else None
            l2 = l2.next if l2 else None

        return dummy.next
```

Listing 9: Python Solution: LC-002 Add Two Numbers